Encrypting Equations

A Group Project Report

SciMathSoc

IIT Kanpur

September 23, 2025

Contents

1 Introduction **3**
- 1.1 Motivation . . . 3
- 1.2 Objective of the Project . . . 4
- 1.3 Outline of the Report . . . 4

2 Mathematical Prerequisites **6**
- 2.1 Galois Field Multiplicaation . . . 6
- 2.2 Binary Exponentiation . . . 8
- 2.3 The Discrete Logarithm Problem . . . 9
- 2.4 Elliptic Curves . . . 10
- 2.5 Elliptic Curves Group Operations . . . 10
- 2.6 Elliptic Curves over $\mathbb{F}_p$ . . . 11
- 2.7 Elliptic curve Discrete Log problem . . . 12
- 2.8 Miller–Rabin Primality Test . . . 13
- 2.9 Basics of Encryption . . . 14

3 RSA encryption **15**
- 3.1 Key Ideas . . . 15
- 3.2 Key generation and distribution . . . 15
- 3.3 Encryption and decryption . . . 16
- 3.4 Implementation . . . 17

4 Diffie–Hellman Key Exchange **18**
- 4.1 Key Ideas . . . 18
- 4.2 Elliptic Curve Diffie–Hellman (ECDH) . . . 19
- 4.3 Implementation . . . 20
  - 4.3.1 Diffie–Hellman Key Exchange . . . 20
  - 4.3.2 Elliptic Curve Diffie–Hellman . . . 21

5 ElGamal Encryption **22**
- 5.1 Key Ideas . . . 22

5.1.1 Key Generation . . . 22
5.1.2 Encryption . . . 23
5.1.3 Decryption . . . 23
5.2 Elliptic Curve ElGamal (ECEG) . . . 23
5.2.1 Key Generation . . . 23
5.2.2 Encryption . . . 24
5.2.3 Decryption . . . 24
5.3 Implementation . . . 24
5.3.1 ElGamal (ECEG) Implementation . . . 24

6 AES Encryption **26**
6.1 Introduction . . . 26
6.2 State Array . . . 26
6.3 XOR Operation (Add Round Key) . . . 26
6.4 Galois Field Multiplication . . . 26
6.5 Substitution-Box . . . 27
6.6 SubBytes . . . 27
6.7 ShiftRows . . . 27
6.8 MixColumns . . . 27
6.9 Round Keys and Key Expansion . . . 27
6.9.1 Round Constants (Rcon) . . . 28
6.9.2 Key Expansion Procedure (128-bit) . . . 28
6.10 Encryption Procedure . . . 28
6.11 Decryption Procedure . . . 29
6.12 Resources . . . 30

7 Lattice Cryptography **31**
7.1 Babai's Algorithm . . . 31
7.2 Cryptosystems Based on Lattices . . . 32
7.2.1 The GGH Cryptosystem . . . 32
7.2.2 Other Lattice Based Cryptosystems . . . 32
7.3 Cryptanalysis of Lattice Based Algorithms . . . 33
7.3.1 Gaussian Lattice Reduction . . . 33
7.3.2 LLL Latice Reduction Algorithm . . . 33

Chapter 1

Introduction

In the modern digital era, the secure transmission of information has become a critical requirement across multiple domains, ranging from personal communications to large-scale financial transactions and national security operations. Cryptography, the mathematical science of securing information, lies at the heart of this challenge. It enables the transformation of readable information (plaintext) into an unintelligible form (ciphertext) that can only be reverted by authorized parties. This project, *Encrypting Equations*, explores and implements several prominent cryptographic algorithms—both classical and modern—covering symmetric, asymmetric, and post-quantum encryption techniques. By delving into the mathematical foundations of these algorithms and providing working implementations, the project aims to bridge the gap between theoretical understanding and practical application.

1.1 Motivation

The increasing reliance on digital communication has amplified the risks associated with data breaches, unauthorized access, and cyberattacks. With advancements in computational capabilities, particularly the looming threat of quantum computing, many traditional encryption schemes are facing potential vulnerabilities. The motivation behind this project stems from the need to understand the mathematical underpinnings of encryption algorithms and to analyze their robustness in different scenarios. By studying and implementing algorithms such as RSA, Diffie–Hellman, ElGamal, AES, and lattice-based cryptography, the project aims to equip learners with both theoretical insight and practical coding skills in modern cryptographic systems. This not only enhances problem-solving capabilities in mathematics and computer science but also prepares for future cryptographic challenges.

1.2 Objective of the Project

The objectives of the *Encrypting Equations* project are:

1. **Mathematical Exploration** – To study the number theory, algebra, and computational mathematics that form the foundation of modern cryptography, including finite fields, elliptic curves, and lattice theory.

2. **Algorithm Implementation** – To develop working implementations of key cryptographic algorithms in programming languages such as C/C++, thereby reinforcing theoretical concepts through practical coding exercises.

3. **Comparative Analysis** – To examine the strengths, weaknesses, and computational efficiency of different encryption techniques, including their resilience to classical and quantum attacks.

4. **Educational Resource** – To compile a structured, well-documented guide that can serve as a reference for students and enthusiasts interested in cryptography.

1.3 Outline of the Report

This report is organized as follows:

- **Chapter 2 – Mathematical Prerequisites:** Covers essential concepts such as Galois Field arithmetic, binary exponentiation, the discrete logarithm problem, elliptic curves, primality testing, and basic encryption principles.

- **Chapter 3 – RSA Encryption:** Explains the RSA algorithm, its mathematical foundations, key generation, encryption/decryption processes, and implementation details.

- **Chapter 4 – Diffie–Hellman Key Exchange:** Discusses the classic DH protocol, its elliptic curve variant (ECDH), and example implementations.

- **Chapter 5 – ElGamal Encryption:** Describes both standard ElGamal and its elliptic curve version (ECEG), detailing key generation, encryption, decryption, and code examples.

- **Chapter 6 – AES Encryption:** Explores the symmetric block cipher AES, including its internal transformations, key expansion, and encryption/decryption procedures.
- **Chapter 7 – Lattice Cryptography:** Introduces lattice-based techniques, algorithms such as Babai's method, the GGH cryptosystem, and cryptanalysis methods like Gaussian and LLL reductions.
- **Conclusion:** Summarizes findings and reflects on the comparative strengths of the studied algorithms, especially in light of future cryptographic challenges.

Chapter 2

Mathematical Prerequisites

2.1 Galois Field Multiplicaation

In AES, the bytes are treated as elements of the Galois Field $GF(2^8)$. Each byte corresponds to a polynomial of degree less than 8 with coefficients in $GF(2)$. For example, the byte `0x57` (binary: 0101 0111) corresponds to the polynomial:

$$x^6 + x^4 + x^2 + x + 1$$

Irreducible Polynomial

Multiplication is carried out modulo the irreducible polynomial used in AES:

$$m(x) = x^8 + x^4 + x^3 + x + 1 \quad \text{(hex: 0x11B)}$$

This ensures the result of the multiplication remains within the field $GF(2^8)$.

Procedure for Galois Field Multiplication

Let a and b be two bytes (i.e., 8-bit values). The goal is to compute:

$$c = a \cdot b \mod m(x)$$

The multiplication is done using the **Russian Peasant Multiplication** algorithm, adapted for $GF(2^8)$:

1. Initialize the result: $\text{res} = 0$

2. Repeat for 8 iterations:

   (a) If the least significant bit (LSB) of b is 1, then:

   $$\text{res} = \text{res} \oplus a$$

   (b) Check if the most significant bit (MSB) of a is 1:

   - If yes:
   $$a = (a \ll 1) \oplus 0x1B$$
   - If no:
   $$a = a \ll 1$$

   (c) Right shift b:
   $$b = b \gg 1$$

Explanation

- The bitwise left shift on a corresponds to multiplying by x in polynomial form.
- If the MSB of a was 1, then the result would be a degree 8 polynomial and must be reduced modulo $m(x)$, which is equivalent to XORing with `0x1B`.
- The final result after 8 steps is the product $a \cdot b \mod m(x)$.

Example

Let $a =$ `0x57`, $b =$ `0x83`:

- Convert to binary: $a = 0101\ 0111$, $b = 1000\ 0011$
- Using the procedure above, the multiplication yields:

$$0x57 \cdot 0x83 = 0xC1$$

Summary

Galois multiplication in AES is not standard integer multiplication. It is a bitwise operation following rules of finite field arithmetic under $GF(2^8)$, involving:

- XOR for addition
- Left shifts and conditional reduction for multiplication
- Modular reduction using irreducible polynomial $x^8 + x^4 + x^3 + x + 1$

2.2 Binary Exponentiation

source to study Binary Exponentiation: Binary Exponentiation by cp-algorithms

Binary exponentiation is a method used to improve the time-complexity of calculating a^n from $O(n)$(using brute-force multiplication) to $O(\log n)$.
The key idea of BE, is to use the binary representation of the exponent. consider the integer n and the exponent a^n, there are exactly $\lfloor \log_2 n \rfloor + 1$ digits in the binary representation of n, hence we need to perform only $O(\log n)$ multiplications considering we know the values of:

$$a, a^2, a^4 ...., a^{2\lfloor \log_2 n \rfloor}$$

These values can be found easily by just squaring the previous term, so we get all of these terms with $O(\log n)$ operations, Finally giving us a time complexity of $O(\log n)$. this method can be used for any kind of group operations, which satisfy associative property, for example: elliptic curves.
Example: Consider calculating the value of 3^{13}, with regular multiplication, it will take $O(n)$ operations to get the final answer. So let's use Binary Exp method to find the answer. Consider the Binary representation of the exponent 13:

$$13 = 1 \cdot 2^0 + 0 \cdot 2^1 + 1 \cdot 2^2 + 1 \cdot 2^3$$

which gives us:

$$3^{13} = 3^1 \cdot 3^4 \cdot 3^8$$

we have to perform only $O(\log n)$ to get the final answer, considering we know the values of each 3^{2k}, which is easy to find by squaring the previous

term:

$$3^1 = 3$$
$$3^2 = 3^1 \cdot 3^1 = 9$$
$$3^4 = 3^2 \cdot 3^2 = 81$$
$$3^8 = 3^4 \cdot 3^4 = 6561$$

The final complexity of this algorithm is $O(\log n)$, we need to compute $\log n$ powers of a, and then do atmost $\log n$ multiplications of these powers.
Note that this method can be used to calculate large scalar multiples of point addition kP in elliptic curves, by considering the binary representation of k.

2.3 The Discrete Logarithm Problem

Consider a group G and an element $g \in G$. The **Discrete Logarithm Problem** states that:

> Consider an element $h \in G$ which is generated by g, find the integer n satisfying
> $$h = g^n$$

The **index**(or logarithm) of h with respect to g is defined as the smallest integer n which satisfies $h = g^n$. It is given by:

$$n = \log_g(h) \quad \text{(or)} \quad n = \text{ind}_g(h)$$

The **DLP** is a widely used hard problem in cryptography, we can find its uses in different algorithms like **Diffie-Hellman key exchange, ELGamal encryption, ECC based algorithms, etc...**
We need to be careful about the Group on which we are trying to use the DLP, as for some groups, DLP is very easy for example:

- $\mathbb{Z}/m\mathbb{Z}$ under addition operation.
- $\mathbb{R}^*$ under multiplication operation.

But for the field $\mathbb{F}_p^*$ under multiplication, the DLP is considered to be hard, with the best known algorithm taking **sub-exponential** time to solve the problem.

2.4 Elliptic Curves

- Elliptic curve is an algebraic curve defined by the Weierstrass formula: $y^2 = x^3 + Ax + B$, where A and B are elements of the field over which the elliptic curve is defined.
- the EC-equation is supposed to have distinct roots, or you can say the Elliptic curve equation should be **non singular**, to find if the curve is non-singular or not, we consider the **Discriminant** of the elliptic curve, which is defined by:

$$\triangle = 4A^3 + 27B^2 \neq 0$$

- We also define an extra point $\mathcal{O}$ at *infinity*, which acts as the identity element of the groups that we construct using the elliptic curve.
- The set of all points we are going to consider is given by:

$$E = \{(x, y) : y^2 = x^3 + Ax + B\} \cup \{\mathcal{O}\}$$

2.5 Elliptic Curves Group Operations

Let's consider two points P and Q over the elliptic curve, the **addition** of points P and Q is defined by extending the line passing through PQ, and then reflecting the third point R at which the line intersects the curve about the x-axis.
The addition law of E has the following properties:

1. $P + \mathcal{O} = \mathcal{O} + P$ for all $P \in E$ (Additive Identity).
2. $P + (-P) = \mathcal{O}$ for all $P \in E$ (Additive Inverse)
3. $P+(Q+R) = (P+Q)+R$ for all $P, Q, R \in E$ (Associative Property)
4. $P + Q = Q + P$ for all $P, Q \in E$ (Commutative Property)

Point Addition Formulas

Let E be an elliptic curve defined over a field with equation:

$$E : y^2 = x^3 + ax + b$$

Let $P = (x_1, y_1)$ and $Q = (x_2, y_2)$ be two points on E, and let $R = P + Q = (x_3, y_3)$ be their sum under the group law. The formulas for point addition are as follows:

- **Case 1:** $P \neq Q$ and $x_1 \neq x_2$

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}, \quad x_3 = \lambda^2 - x_1 - x_2, \quad y_3 = \lambda(x_1 - x_3) - y_1$$

- **Case 2:** $P = Q$ (point doubling) and $y_1 \neq 0$

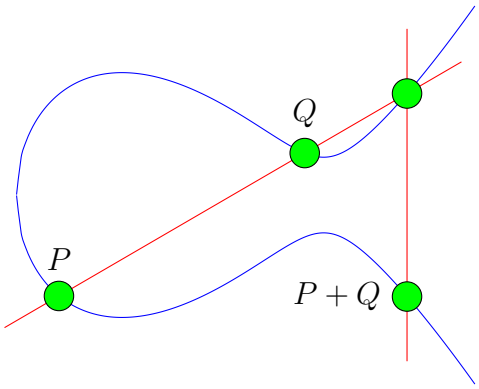
$$\lambda = \frac{3x_1^2 + a}{2y_1}, \quad x_3 = \lambda^2 - 2x_1, \quad y_3 = \lambda(x_1 - x_3) - y_1$$

- **Case 3:** $P = -Q$ or $P = Q$ and $y_1 = 0$

$$P + Q = \mathcal{O}$$

(the point at infinity, i.e., the identity element)

Here, λ denotes the slope of the tangent or secant line used in the addition process.



2.6 Elliptic Curves over $\mathbb{F}_p$

consider the field $\mathbb{F}_p$ which is defined by:

$$\mathbb{F}_p = \{0, 1, 2, 3...., p - 1\}$$

The elliptic curve group defined over $\mathbb{F}_p$ is given by:

$$E(\mathbb{F}_p) = \{(x, y) : y^2 = x^3 + Ax + B \pmod{p} \quad \text{with } A, B \in \mathbb{F}_p\} \cup \{\mathcal{O}\}$$

The addition formulas which were given earlier can be used with the usage of $\pmod{p}$.
Let $P = (x_1, y_1), Q = (x_2, y_2) \in E(\mathbb{F}_p)$. Write $P + Q = R = (x_3, y_3)$. Then:

Inverse:

$$-P \;=\; (x_1,\, -y_1) \pmod p, \quad P + (-P) \;=\; \mathcal{O}.$$

Addition $(P \neq \pm Q)$**:**

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1} \pmod p, \tag{2.1}$$

$$x_3 = \lambda^2 - x_1 - x_2 \pmod p, \tag{2.2}$$

$$y_3 = \lambda\,(x_1 - x_3) - y_1 \pmod p. \tag{2.3}$$

Doubling $(P = Q,\ y_1 \neq 0)$**:**

$$\lambda = \frac{3x_1^2 + A}{2\,y_1} \pmod p, \tag{2.4}$$

$$x_3 = \lambda^2 - 2\,x_1 \pmod p, \tag{2.5}$$

$$y_3 = \lambda\,(x_1 - x_3) - y_1 \pmod p. \tag{2.6}$$

Special cases:

- If $P = \mathcal{O}$ then $P + Q = Q$.
- If $P = (x_1, 0)$ then $P + P = \mathcal{O}$ (because the tangent is vertical).

2.7 Elliptic curve Discrete Log problem

The elliptic curve DLP can be stated as:

Let E be an elliptic curve defined over $\mathbb{F}_p$.

$$E : y^2 = x^3 + Ax + B \quad A, B \in \mathbb{F}_p.$$

Let S and T be points in $E(\mathbb{F}_p)$, Find an integer m, such that:

$$T = mS$$

We know that m is defined to be the **index**(or **logarithm**) of T with respect to S.
There are different methods to solve the ECDLP, some of them are the exhaustive search method(brute-force method), collision-search method, and most importantly, **Pollard's ρ - method**. These methods are not going to be discussed here for now. Here are some sources to study Pollard's ρ - method:

- Pollard's rho algorithm - cp-algorithms
- Pollard's rho algorithm - Wikipedia

2.8 Miller–Rabin Primality Test

source: Stanford Number Theory notes
The Miller–Rabin test is an efficient probabilistic algorithm to detect compositeness of an odd integer, note that this test doesn't guarantee that your number is prime or not. $n > 2$. Write

$$n - 1 = 2^s\, q,$$

with q odd. For a randomly selected base $a \in \{1, \ldots, n-1\}$ consider the sequence

$$a^q \bmod n,\ a^{2q} \bmod n,\ a^{2^2 q} \bmod n,\ \ldots,\ a^{2^s q} \bmod n = a^{n-1} \bmod n.$$

If n is prime then this sequence either begins with 1, or the first term that differs from 1 must be -1. Finding a term that contradicts this behaviour proves that n is composite. If the sequence behaves as for a prime, we say that n is a *strong probable prime* to base a.

Single-round procedure.

1. Compute integers s, q with $n - 1 = 2^s q$ and q odd.
2. Choose a random base $a \in [2, n-2]$.
3. Compute $x \leftarrow a^q \bmod n$.
4. If $x \equiv 1 \pmod n$ or $x \equiv -1 \pmod n$, the round *passes*.
5. Otherwise, for $i = 1, \ldots, s-1$ set $x \leftarrow x^2 \bmod n$. If at any step $x \equiv -1 \pmod n$, the round passes.
6. If none of the above conditions hold, declare n **composite**.

Error bound. For any composite n the probability that a *random* base a causes Miller–Rabin to pass is at most $1/4$. Repeating k independent rounds reduces the error probability to at most 4^{-k}.

Implementation notes.

- Perform a few trial divisions by small primes before Miller–Rabin to eliminate simple composites cheaply.

- Use fast modular exponentiation (binary exponentiation) for computing $a^q \bmod n$.
- For deterministic checks on inputs below certain bounds, use fixed base sets known to be sufficient for those bounds.

2.9 Basics of Encryption

Encryption is the process of transforming plaintext into ciphertext so that unauthorized parties cannot read it. There are two main types of encryption:

1. Symmetric (Private-Key) Encryption.

- A single secret key K is shared between sender and receiver.
- Encryption: $C = \mathcal{E}_K(M)$.
- Decryption: $M = \mathcal{D}_K(C)$.
- Examples: AES, ChaCha20, DES.
- Very fast; key-distribution is the main challenge.

2. Asymmetric (Public-Key) Encryption.

- Key pair $(K_{\text{pub}}, K_{\text{priv}})$ with K_{pub} publicly known.
- Encryption: $C = \mathcal{E}_{K_{\text{pub}}}(M)$.
- Decryption: $M = \mathcal{D}_{K_{\text{priv}}}(C)$.
- Examples: RSA, ElGamal, ECC-based schemes.
- Solves the key-distribution problem, but slower than symmetric ciphers.

Chapter 3

RSA encryption

Sources: RSA encryption by Brilliant and RSA cryptosystem by Wikipedia.

3.1 Key Ideas

RSA is an example of public key/asymmetric cryptography. There are four key steps to the RSA algorithm.

1. key generation
2. key distribution
3. encryption
4. decryption

3.2 Key generation and distribution

- There are two keys that are used in RSA, the **public key** and the **private key**. The public key is shared while the private key is only present with the receiver. To create the public key, we first have to choose two really big prime numbers. Usual size of the numbers is around 1024 bits (around 300 digits each), lets say these numbers are p and q. We consider their product $n = pq$, as the first half of the public key.
- The second half of the public key is obtained by using the Euler-totient-function

$$\phi(n) = (p-1) \cdot (q-1)$$

- We find another number e which is **relatively prime** to $\phi(n)$, usually we choose $e = 2^{16} + 1$ or $e = 3$, we need e to have short-bit length and small hamming weight, e **forms the second part of our public key.**

- Calculate the multiplicative inverse of $e \pmod{n}$, assume d is the multiplicative inverse of e, then we have:

$$d \cdot e \equiv 1 \pmod{n}$$

- d is our **private key**, this is supposed to be kept secret only with the recipient.

- Alice shares her public key with Bob through a reliable channel, but doesn't share her private key.

$$\text{Alice} \xrightarrow{\text{Public key}} \text{Bob}$$

- Let's say Bob has to send an encrypted message to Alice, Bob needs to know the public key of Alice to encrypt his message, and then he shares his encrypted message to Alice, which is then decrypted by Alice using her private key.

$$\text{Bob} \xrightarrow{\text{Encrypted message}} \text{Alice}$$

3.3 Encryption and decryption

- Let's say Bob has a text-message that he wants to encrypt, firstly he needs to convert it into number-format by using common methods like ASCII.

- Assuming he converted his message into a number m, we need to make sure that m is smaller than n, i.e $m < n$, because if it is not, then the message will be lost when modulo is taken. Hence if m is greater than n, then divide it into smaller blocks, and send the message block wise.

- Once we have the message m, to encrypt it, we need to take modulo exponent of m with public key e:

$$m^e \equiv c \pmod{n}$$

  where c is our new encrypted message (also known as **ciphertext**).

- This ciphertext is now transferred to Alice, who takes the modular exponent of the ciphertext c with the private key d.

$$c^d \equiv m \pmod{n}$$

  through which Alice gets the original message back.

3.4 Implementation

A very skeletal code for the implementation of RSA is given here, note that the implementation uses the GNU-MP library:

- RSA-GMP.c

The output of the code is given by(note that we are printing every value only for demonstration):

```
Primality test passed I guess.
p = 90344467931018478790369878185660823024745220039813
q = 69388692278459310636414277690654863246368852525374 79

String_p: 90344467931018478790369878185660823024745220039813
String_q: 69388692278459310636414277690654863246368852525374 79
the value of n is equal to:
 ↪ 62688844843265767309785505463840257866703235290347409601332
90445087285770801685422167807886596546514 27
phi(n) is given by:
 ↪ 62688844843265767309785505463840257866703235290346706679963
32675591861452825212910696331270291820741 36
e is equal to: 100000000000000000 01
gcd of phi(n) and e is equal to: 1
d is equal to:
 ↪ 43960534538922430432224685465307919211389158056456836091 95
30371460078820870254707992788128236827797969
Enter maximum string length (make sure len(msg)<n): 10
Enter your string (up to 10 characters): 1234567890
You entered: "1234567890"

your encrypted message is:
 ↪ 18837695886046200134476153711111074180335501375897805 82
75426265198927897319336758320824329325608641288
your decrypted message is: 1234567890
```

Chapter 4

Diffie–Hellman Key Exchange

sources: Diffie–Hellman Key Exchange by Wikipedia and
Elliptic-Curve Diffie–Hellman by Wikipedia.

4.1 Key Ideas

Diffie–Hellman Key Exchange (DHKE) is an example of a public-key algorithm used for establishing a shared secret over an insecure channel. It is **not** an encryption algorithm itself, but rather a method for two parties to agree upon a secret key that can then be used for symmetric encryption.

The DHKE process consists of the following steps:

1. **Parameter agreement:** Both parties (Alice and Bob) agree on a set of public parameters, which can be sent over an insecure channel:

   - **Prime modulus** p: a large prime number, typically at least 2048 bits for modern security.
   - **Primitive root modulo** p g: an integer whose powers modulo p generate all integers from 1 to $p - 1$.

   These public parameters (p, g) form the basis for the exchange.

2. **Secret key generation:** Each party independently selects a large random integer as their private key:

   - Alice's secret key: a such that $1 < a < p - 1$
   - Bob's secret key: b such that $1 < b < p - 1$

   These private keys are never shared.

3. **Public value calculation and exchange:** Using the private keys and public parameters, each party computes a public value:

   - Alice computes $A \equiv g^a \pmod{p}$ and sends A to Bob.
   - Bob computes $B \equiv g^b \pmod{p}$ and sends B to Alice.

   An eavesdropper may see (p, g, A, B) but cannot efficiently determine a or b.

4. **Shared secret computation:** Each party uses their private key and the other party's public value to compute the shared secret:

$$S \equiv B^a \pmod{p} = g^{ab} \pmod{p}$$

$$S \equiv A^b \pmod{p} = g^{ab} \pmod{p}$$

   Since $g^{ab} = g^{ba}$, both parties obtain the same shared secret S, which can then be used for symmetric encryption.

4.2 Elliptic Curve Diffie–Hellman (ECDH)

Elliptic Curve Diffie–Hellman (ECDH) is a more modern and efficient variant of DHKE that uses elliptic curve mathematics to achieve the same goal with smaller key sizes.

1. **Parameter agreement and curve selection:**

   - **Elliptic curve:** defined by

$$y^2 \equiv x^3 + ax + b \pmod{p}$$

     where a and b are constants and p is a large prime specifying the finite field.
   - **Base point** G: a publicly agreed-upon generator point on the curve.

2. **Secret key generation:**

   - Alice's private key: d_A
   - Bob's private key: d_B

3. **Public key calculation and exchange:**

   - Alice computes $P_A = d_A \cdot G$ and sends P_A to Bob.

- Bob computes $P_B = d_B \cdot G$ and sends P_B to Alice.

An eavesdropper cannot compute d_A or d_B from P_A or P_B due to the elliptic curve discrete logarithm problem.

4. **Shared secret computation:**

   - Alice computes $S = d_A \cdot P_B = (d_A d_B) \cdot G$
   - Bob computes $S = d_B \cdot P_A = (d_B d_A) \cdot G$

   Both arrive at the same S, which can be used as a symmetric encryption key.

4.3 Implementation

4.3.1 Diffie–Hellman Key Exchange

A skeletal C++ implementation of DHKE (using a large number arithmetic library):

- String_Diffie-Hellman_Key_Exchange.cpp

Example output (values printed for demonstration):

```
Safe-Prime is
32317006071311007300714876688669951960444102669715484032130345427 52...
Base of power is
2
First person chose the key
1000
Second person chose the key
2000
First person sent message
10715086071862673209484250490600018105614048117055336074437503883 70...
Second person sent message
11481306952742545242328332011776819840223177020886952004776427368 25...
Final Secret Key is
21278422785917832385665260277401504149862520152408187079786504018 23...
```

4.3.2 Elliptic Curve Diffie–Hellman

A very skeletal code for the implementation of ECDH is given here, note that the implementation uses the GNU-MP library:

- ECDH.c

Example output:

```
Your Private keys are:
 ↪  640e9d40c52168e21b6462e731bc58d846d594fea2da7b9ec8b66a76af638e6b,
2d3881594851c9e09101f93588b7db8ed23eac88adf6ef566bacc519191a138c

The private keys are 256-bit Randomly generated numbers.
Note that we are printing the private keys here only for
 ↪  presentation.
In a real application, you would never print or expose your
 ↪  private keys.

the elliptic curve is given by: y^2 = x^3 + 5 x + 87 (mod 524287)
the value of the generator is: g = (3, bb36)
the values of our public keys are: ag = (28179, 5b1a), bg =
 ↪  (736fb,64254)
the value of the points abg and bag is: abg = (57a2f, 53a77), bag
 ↪  = (57a2f, 53a77)
Success: abg == bag, shared secret matches
```

Chapter 5

ElGamal Encryption

sources: ElGamal encryption by Wikipedia

5.1 Key Ideas

ElGamal is a public-key encryption system that, similar to the Diffie–Hellman Key Exchange (DHKE), is built upon the mathematical properties of modular exponentiation. It allows a sender to encrypt a message for a specific recipient using their public key. Unlike DHKE (which is for key agreement), ElGamal is a full-fledged encryption algorithm.

The key steps of the ElGamal encryption process are as follows:

1. Key Generation
2. Encryption
3. Decryption

5.1.1 Key Generation

- **Parameter agreement:** Choose a large prime modulus p and a primitive root modulo p, denoted g. These parameters are public.
- **Private key generation:** The recipient selects a random integer x such that $1 < x < p - 1$. This x is kept secret.
- **Public key calculation:** Compute

$$h \equiv g^x \pmod{p}.$$

The recipient publishes the public key tuple (p, g, h).

5.1.2 Encryption

- **Random secret selection:** The sender chooses a random ephemeral integer k with $1 < k < p - 1$.
- **Ciphertext calculation:** For a plaintext integer m with $0 \leq m < p$, compute

$$c_1 \equiv g^k \pmod{p}, \qquad c_2 \equiv m \cdot h^k \pmod{p}.$$

  The ciphertext is the pair (c_1, c_2).

5.1.3 Decryption

- **Shared secret computation:** Using the private key x,

$$s \equiv c_1^x \pmod{p}.$$

- **Message recovery:** Compute the modular inverse $s^{-1} \pmod{p}$ and recover

$$m \equiv c_2 \cdot s^{-1} \pmod{p}.$$

  This works since $s = (g^k)^x = g^{kx}$ cancels the blinding factor in $h^k = (g^x)^k = g^{kx}$.

5.2 Elliptic Curve ElGamal (ECEG)

Elliptic Curve ElGamal (ECEG) is the elliptic-curve analogue of ElGamal. It provides probabilistic encryption using points on an elliptic curve and achieves comparable — or better — security with much smaller keys.

5.2.1 Key Generation

- **Parameter agreement and curve selection:** Choose a prime p and an elliptic curve defined by

$$y^2 \equiv x^3 + ax + b \pmod{p},$$

  together with a publicly agreed base point G of large order.
- **Private key selection:** The recipient chooses a private key d uniformly at random from the valid range (order of G).
- **Public key calculation:** Compute the public key point

$$P_B = d \cdot G.$$

5.2.2 Encryption

- **Message conversion:** Map the plaintext to a point M on the chosen curve (message encoding).
- **Random secret selection:** Choose a random ephemeral integer k.
- **Ciphertext calculation:** Compute the point-pair

$$C_1 = k \cdot G, \qquad C_2 = M + k \cdot P_B.$$

  The ciphertext is (C_1, C_2).

5.2.3 Decryption

- **Shared secret computation:** Compute the shared secret point

$$S = d \cdot C_1.$$

- **Plaintext recovery:** Recover the message point by

$$M = C_2 - S,$$

  then convert the point M back to the original plaintext.

5.3 Implementation

5.3.1 ElGamal (ECEG) Implementation

A very skeletal code for the implementation of ECEG is given here, note that the implementation uses the GNU-MP library:

- `ECEG.c`

The output of the code is given below (note that we print values only for demonstration purposes):

```
1 Your private key is given by: 5bbf6
2 Please note that we are printing the private key here only for
  ↪ presentation.
3 In a real application, you would never print or expose your
  ↪ private keys.
4
```

```
your public key is given in the format (p, g, h): (524287, (3,
↪  47926), (59766, 136005))
 Please enter your message to encrypt(make sure your message is a
 ↪  number and is less than 524287): 524286
The point corresponding to your message is (524286, 9)
your message x-coordinate shifted by 0
The first ciphertext is given by: (211684, 329301)
The second ciphertext is given by: (84010, 249850)
The value of s is given by: (125375, 78596)
the value of s_inv is (125375, -78596)
The decrypted message is given by: (524286, 9)
The decrypted message x-coordinate shifted back to original by
↪  (count) 0 is: 524286
```

Chapter 6

AES Encryption

6.1 Introduction

The Advanced Encryption Standard (AES) is a symmetric block cipher adopted as a standard by NIST in 2001. It encrypts data in fixed blocks of 128 bits using cipher keys of 128, 192, or 256 bits. AES is secure, fast, and suitable for hardware and software implementation.

6.2 State Array

The AES algorithm processes data as a 4×4 byte matrix known as the **State**. Each byte is an element of the finite field $\mathbb{F}_{2^8}$.

6.3 XOR Operation (Add Round Key)

AES uses XOR to combine the State with the Round Key.

The XOR is a fast and reversible bitwise operation. It is used in the `Add Round Key` step.

6.4 Galois Field Multiplication

AES uses multiplication in the Galois Field ($GF(2^8)$), with the irreducible polynomial:

$$x^8 + x^4 + x^3 + x + 1$$

This defines how bytes are multiplied.

- Common constants used: 1, 2, 3 (in hexadecimal: 0x01, 0x02, 0x03)

- Multiplication is performed modulo the above polynomial.

6.5 Substitution-Box

It is a look-up table that helps in transforming one state into another new state in the encryption process.

6.6 SubBytes

Each byte of the State is substituted using a nonlinear S-box derived from the multiplicative inverse in $\mathbb{F}_{2^8}$.

6.7 ShiftRows

AES cyclically shifts each row in the State:

- Row 0: no shift
- Row 1: left shift by 1 unit
- Row 2: left shift by 2 units
- Row 3: left shift by 3 units

6.8 MixColumns

Each column is treated as a 4-byte vector and multiplied by a fixed matrix over $GF(2^8)$, given by:

$$\begin{bmatrix} d_0 \\ d_1 \\ d_2 \\ d_3 \end{bmatrix} = \begin{bmatrix} 2 & 3 & 1 & 1 \\ 1 & 2 & 3 & 1 \\ 1 & 1 & 2 & 3 \\ 3 & 1 & 1 & 2 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

6.9 Round Keys and Key Expansion

AES expands the cipher key into an array of words. So, it gets expanded to 11 rounds from 10 rounds. Each round uses 4 words (16 bytes). Each round key expansion uses:

- **RotWord**: Rotates a 4-byte word left by 1 unit.
- **SubWord**: Substitutes a 4-byte word using S-box on each byte.
- **Rcon**: Denotes round constants which are added to the subword.

6.9.1 Round Constants (Rcon)

Round constants are derived from powers of 2 in $GF(2^8)$.
They are defined recursively as:

$$\begin{aligned} rc(1) &= 1 \\ rc(i) &= 2 \cdot rc(i-1), \quad \text{if } rc(i-1) < 0x80 \\ rc(i) &= 2 \cdot rc(i-1) \oplus 0x11B, \quad \text{if } rc(i-1) \geq 0x80 \end{aligned}$$

Each Rcon is XOR'ed to the first byte of the round key.

6.9.2 Key Expansion Procedure (128-bit)

Each round key is passed through **RotWord**, then passed through **SubWord**, and finally **Rcon** is added.
Columns of next round key are obtained through addition of the previous column of the same round key and the corresponding column in the previous round key.

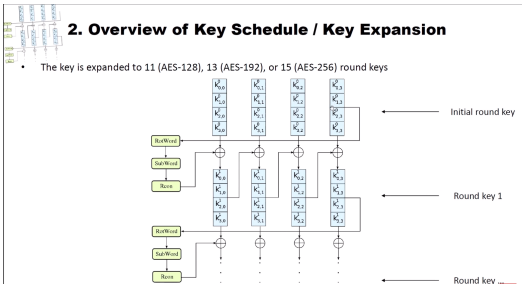


Figure 6.1: AES-128 Algorithm

6.10 Encryption Procedure

AES encryption consists of the following steps for a 128-bit key:

1. **Initial Round**:
   - AddRoundKey(State, RoundKey)

2. **Main Rounds (9 rounds)**:
   For round $r = 1$ to 9:
   - SubBytes(State)
   - ShiftRows(State)
   - MixColumns(State)
   - AddRoundKey(State, RoundKey)
3. **Final Round (10th)**:
   - SubBytes(State)
   - ShiftRows(State)
   - AddRoundKey(State, RoundKey)
4. The output is the ciphertext (final State matrix converted to bytes).

6.11 Decryption Procedure

The decryption process of AES is the inverse of encryption and consists of:

1. **Initial Round (10th)**:
   - AddRoundKey(State, RoundKey)
2. **Main Rounds (9 rounds)**:
   For round $r = 9$ to 1:
   - InvShiftRows(State)
   - InvSubBytes(State)
   - AddRoundKey(State, RoundKey)
   - InvMixColumns(State)
3. **Final Round (0th)**:
   - InvShiftRows(State)
   - InvSubBytes(State)
   - AddRoundKey(State, RoundKey)
4. The output is the original plaintext.

The inverse operations (`InvSubBytes`, `InvShiftRows`, `InvMixColumns`) use corresponding inverse S-box and transformations in $GF(2^8)$.

6.12 Resources

AES-128 Algorithm
AES – The Advanced Encryption Standard Explained
AES – Key Schedule/Key Expansion Explained

Chapter 7

Lattice Cryptography

Lattice based cryptographic techniques are an example of techniques which are resistant to quantum computing attacks. Hence, they form an important and popular example of post-quantum cryptographic techniques. They are based on the concept of lattices in linear algebra.

Definition-1

A lattice L is a set given by

$$L = \{a_1\mathbf{v_1} + a_2\mathbf{v_2} + a_3\mathbf{v_3}... + a_n\mathbf{v_n} \mid a_1, a_2...., a_n \in \mathbb{Z}\}$$

with $\mathbf{v_1}, \mathbf{v_2}, \mathbf{v_3}..., \mathbf{v_n}$ being linearly independent vectors.

Usually, lattice based cryptography is based on the problem of finding the closest lattice point to a given vector. This is called the CVP(Closest Vector Problem). Another related problem is the SVP(Shortest Vector Problem) which involves finding the shortest vector of the lattice.

7.1 Babai's Algorithm

Babai's closest vertex algorithm is an algorithm designed to give an approximate solution to the CVP. Babai's algorithm needs to have an 'appropriate basis' for the lattice to have a good approximate solution to the problem. This appropriate basis must be reasonably orthogonal. Suppose we have a lattice L and an appropriate basis for L which is reasonably orthogonal $\mathbf{v_1}, \mathbf{v_2}..., \mathbf{v_n}$ and we need to solve the CVP for $\mathbf{w}$ then Babai's algorithm is as follows.

Babai's Algorithm

Write $\mathbf{w}$ as $\mathbf{w} = a_1\mathbf{v_1} + a_2\mathbf{v_2} + a_3\mathbf{v_3}... + a_n\mathbf{v_n}$
Set $t_i = [a_i]$ for $i = 1$ to n where $[a_i]$ denotes the closest integer to a_i.
Return $\mathbf{v} = t_1\mathbf{v_1} + t_2\mathbf{v_2} + t_3\mathbf{v_3}... + t_n\mathbf{v_n}$

7.2 Cryptosystems Based on Lattices

One of the earliest cryptosystems based on lattice problems was the GGH(Goldreich, Goldwasser and Halevi) cryptosystem.

7.2.1 The GGH Cryptosystem

Alice begins with linearly independent vectors

$$\mathbf{v_1}, \mathbf{v_2}, \mathbf{v_3}...\mathbf{v_n} \in \mathbb{Z}^n$$

such that they are reasonably orthogonal. Let V be the matrix with these vectors as its rows. It can be shown that left multiplying V with another integer n-by-n matrix W with det(W) = 1 or -1 yields another basis for the lattice L. Now, Alice chooses a random W satisfying the above property. A way to do this is to randomly select and multiply many elementary matrices. Using this Alice gets another basis $U = WV$ for the lattice L. She ensures that this basis is as non-orthogonal as possible by selecting another W if it isn't. This basis U now becomes the public key and the basis V is Alice's private key. Bob then uses the matrix W as follows to encode the plaintext $\mathbf{m}$.(which is in the form of a binary vector)

$$\mathbf{e} = \mathbf{m}W + \mathbf{r}$$

where $\mathbf{e}$ is the ciphertext and $\mathbf{r}$ is a randomly chosen small peturbation vector. Alice now uses her private key to find $\mathbf{m}$ using the fact that $\mathbf{m}W$ is the solution to the CVP for $\mathbf{e}$.

7.2.2 Other Lattice Based Cryptosystems

Some of the more modern cryptosystems based on lattices include NTRU-Encrypt which is based on convolutional polynomial rings and Kyber.

7.3 Cryptanalysis of Lattice Based Algorithms

7.3.1 Gaussian Lattice Reduction

An attack on the above cryptosystems would require us to find a basis such that Babai's algorithm can be applied successfully to solve the CVP. Gaussian Lattice Reduction algorithm is one way to find a good basis given a bad one for the same lattice.

Gaussian Lattice Reduction

Let $L \subset \mathbb{R}^2$ be a 2 dimensional lattice with basis vectors $\mathbf{v_1}$ and $\mathbf{v_2}$, then the following algorithm terminates and yields a good basis for L.
Loop while True:
If $|| \mathbf{v_2} || < || \mathbf{v_1} ||$, swap $\mathbf{v_1}$ and $\mathbf{v_2}$.
Compute $m = [\mathbf{v_1} \cdot \mathbf{v_2} / || \mathbf{v_1} ||^2]$
Replace $\mathbf{v_2}$ with $\mathbf{v_2} - m\mathbf{v_1}$
If $|| \mathbf{v_2} || > || \mathbf{v_1} ||$, terminate the loop

It can be shown that after the above algorithm terminates, $\mathbf{v_1}$ is the shortest vector in the lattice L and that the basis $\{\mathbf{v_1}, \mathbf{v_2}\}$ is quite orthogonal.

7.3.2 LLL Latice Reduction Algorithm

This is an extension of the Gaussian Reduction Algorithm to higher dimensions. It takes a 'bad' basis of a lattice L and transforms it into a basis of shorter and more orthogonal vectors. The shortest vector of the basis after the algorithm terminates is an approximate solution to the SVP. This can be used to attack lattice based cryptosystems and thus the number of dimensions used in those cryptosystems must be so that the LLL algorithm becomes infeasible to compute a good basis of the lattice.